Banco de dados

Sumário

Conceito de banco de dados relacional

Introdução ao SQLite

Conhecendo ferramentas SGBD

- *DBEver*

ORM

- Conceito
- Usabilidade
- Cuidados

Sumário

Conhecendo o Prisma ORM

Instalando o Prisma ORM no nosso projeto

Configurando conexão com banco de dados através do Prisma

Migrations

- Cuidados ao desenvolver (npx migrate)

Relembrando

Banco de dados Relacional

Estruturado em tabelas

Usam chaves primárias e estrangeiras para relacionar tabelas

Utilizam SQL

Possuem schema fixo (campos definidos)

MySQL, Postgres, SQLite, MariaDB, Oracle, SQL Server…

Banco de dados Não Relacional

Podem seguir esquema de arquivos, colunar, chave-valor, grafos…

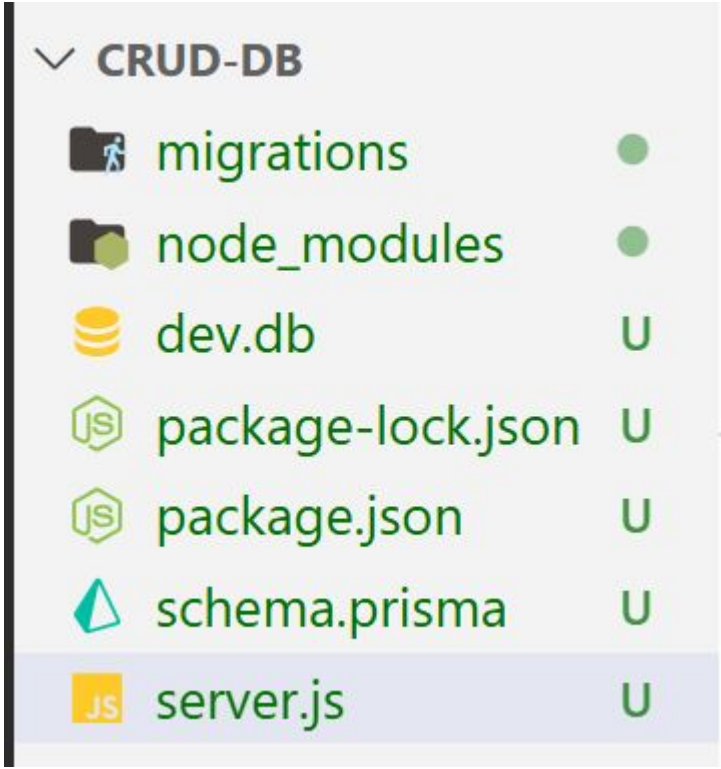
Permitem schemas dinâmicos (campos variam)

Otimizado para grande volumes de leitura e escrita

Cassandra, DynamoDB, MongoDB, Redis, Neo4j

SQLite

- Banco de dados baseado em arquivo
- Fácil manipulação
- Fácil sintaxe (SQL)
- OpenSource (código aberto)



Instalando SQLite

sqlite.org

Latest Release

Version 3.50.4 (2025-07-30). Download Prior Releases

Usando WINGET

Botão Windows

Powershell

digitar: ***winget install SQLite.SQLite***

Verificando a versão:

Feche o terminal e abra um novo

Digite o comando **sqlite3 - -version**

Caso falhe o WINGET - Método Manual

Manual de instalação de forma manual está no GitHub, nos arquivos auxiliares

SGBD - Sistema Gerenciador de Banco de Dados

Intermediário entre o banco de dados e o usuário que o utiliza

Gere a criação, acesso, manipulação, segurança e organização dos dados

Interface bonitinha pra você usar o banco de dados

Instalando o DBEver

https://dbeaver.io/download/

Download

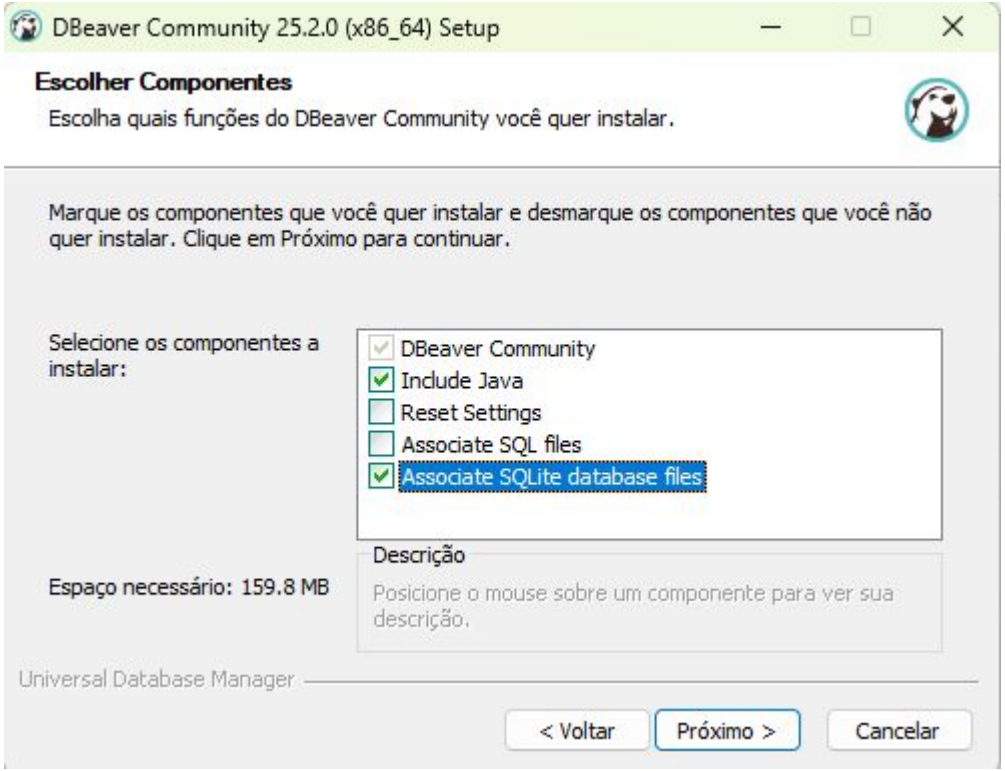
DBeaver Community 25.2.0

Released on August 31st 2025 (Milestones).
It is free and open source (license).
Also you can get it from the GitHub mirror.
System requirements.

Windows

- **Windows (installer)**
- Windows (zip)

Importante!!


DBeaver Community 25.2.0 (x86_64) Setup
Escolher Componentes
Escolha quais funções do DBeaver Community você quer instalar.
Marque os componentes que você quer instalar e desmarque os componentes que você não quer instalar. Clique em Próximo para continuar.
Selecione os componentes a instalar:
DBeaver Community
Include Java
Reset Settings
Associate SQL files
Associate SQLite database files
Descrição
Posicione o mouse sobre um componente para ver sua descrição.
Espaço necessário: 159.8 MB
Universal Database Manager
< Voltar
Próximo >
Cancelar

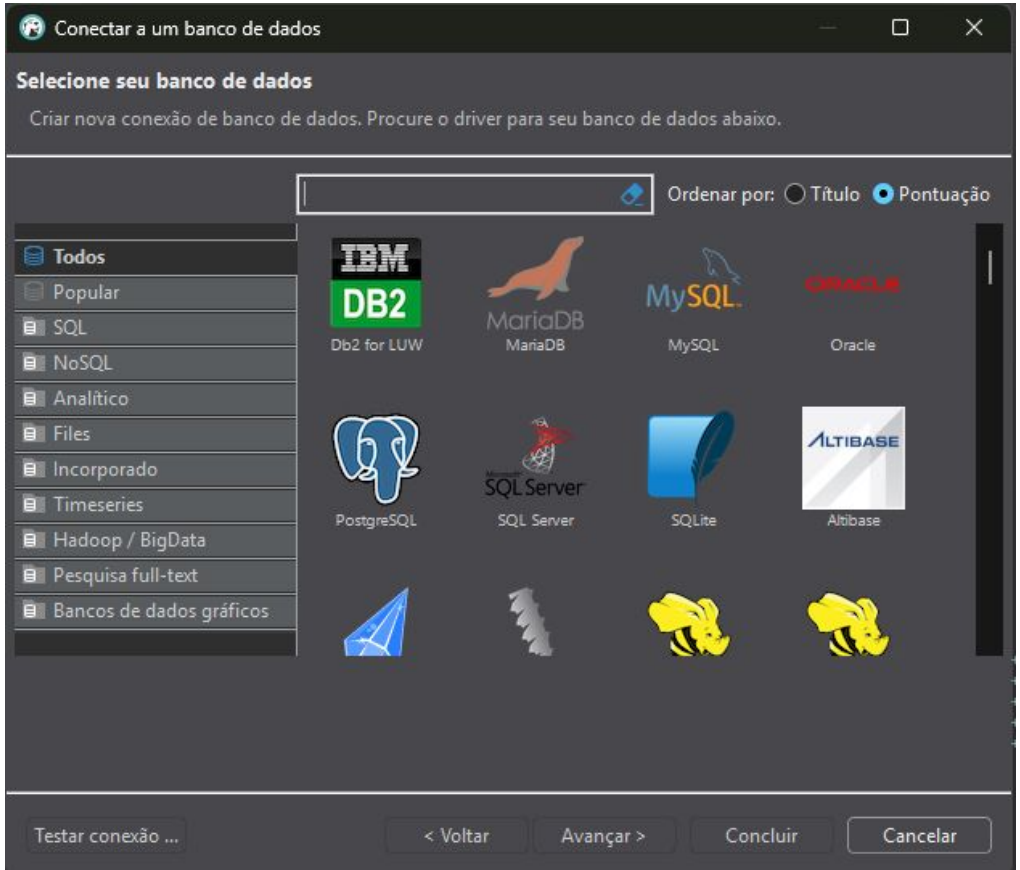
DBeaver
EXPORT
IMPORT
COMMUNITY
Universal Database Manager

Conectando a um banco de dados com o DBEver

Nova conexão

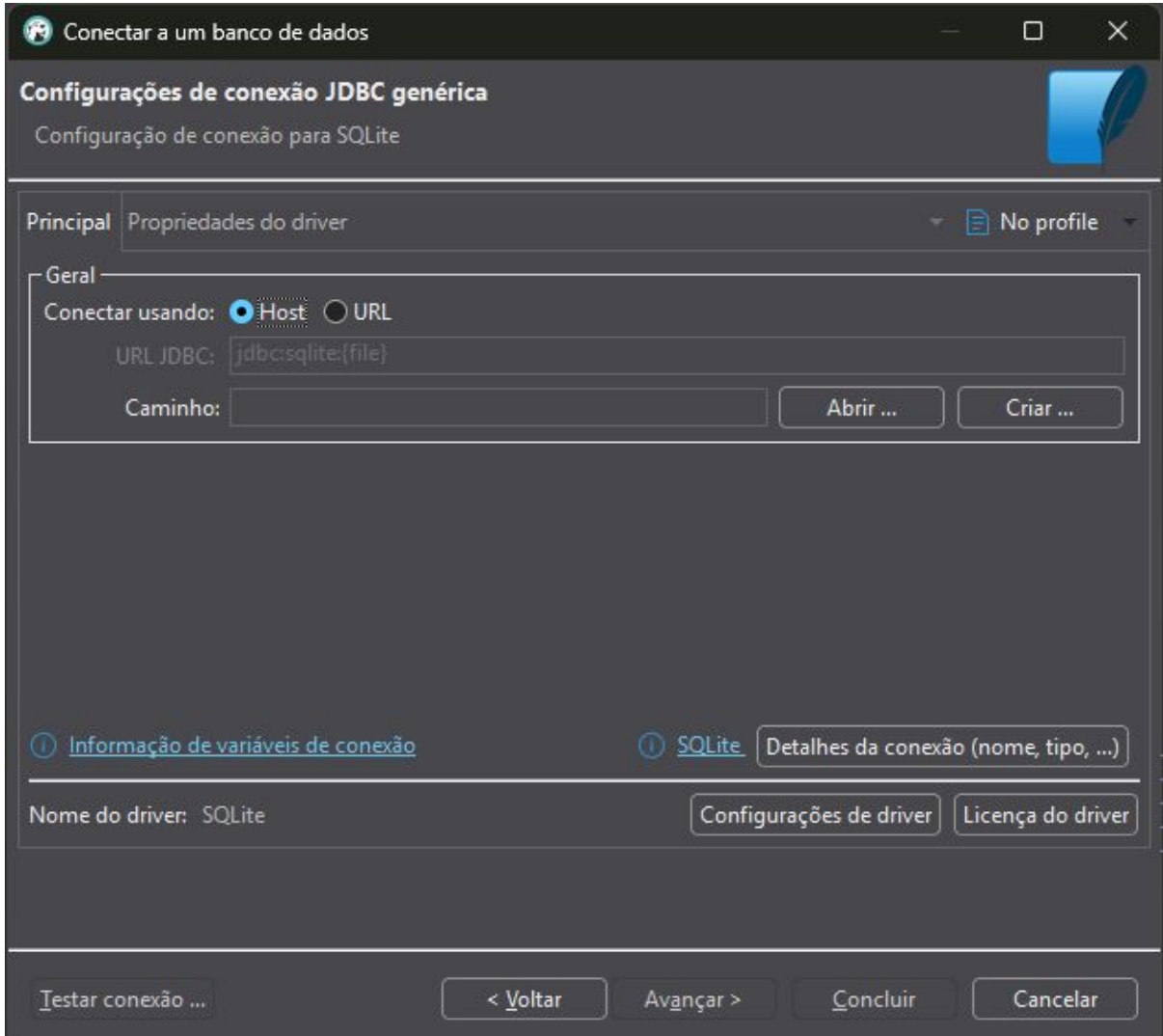
Escolher o tipo

de banco (SQLite)

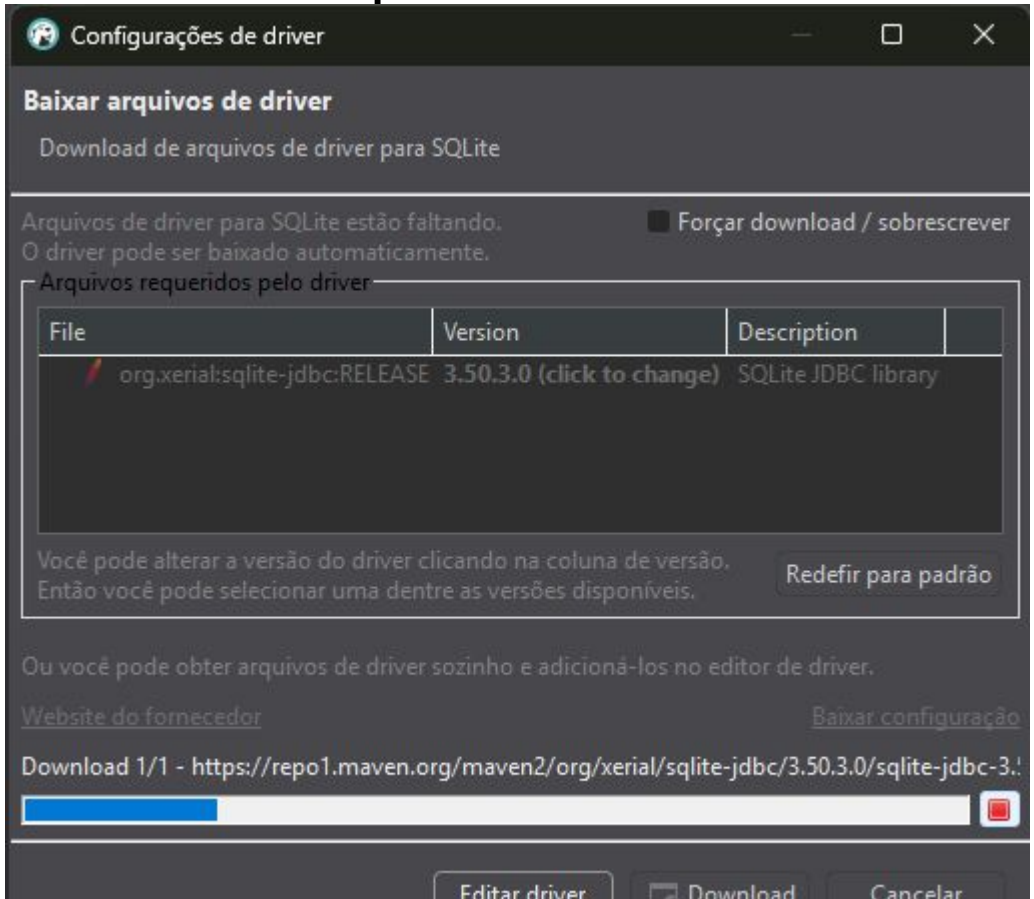


Selecionar um arquivo de banco de dados

Vamos criar um como exemplo



Caso se depararem com isso:



Basta clicar em “Donwload” que os pacotes serão baixados

ORMs

O que é

Object-Relacional Mapping (Mapeamento objeto-relacional**)**

É uma espécie de “ponte” entre a programação em si e o banco de dados

Permite que você utilize funções pré-escritas ao invés de escrever um SQL puro

Para que serve?

- Abstração de SQL: você escreve código, programa, ao invés de SQL
- Possuem proteção automatica contra *sql injection*
- Desenvolvimento mais rápido
- Facilita a mudança entre diferentes bancos de dados

Que cuidados eu devo tomar? Quais as desvantagens?

- **Performance inferior:** Pode gerar queries demoradas
- **Queries complexas:** algumas consultas avançadas (muitos joins) podem acabar se tornando mais complexas usando ORM do que usando SQL
- **Curva de aprendizagem:** cada ORM tem sua própria linguagem e suas próprias funções
- **Dependência:** seu código fica acoplado ao ORM

Como eu sei o recurso que cada um tem? Preciso decorar tudo?

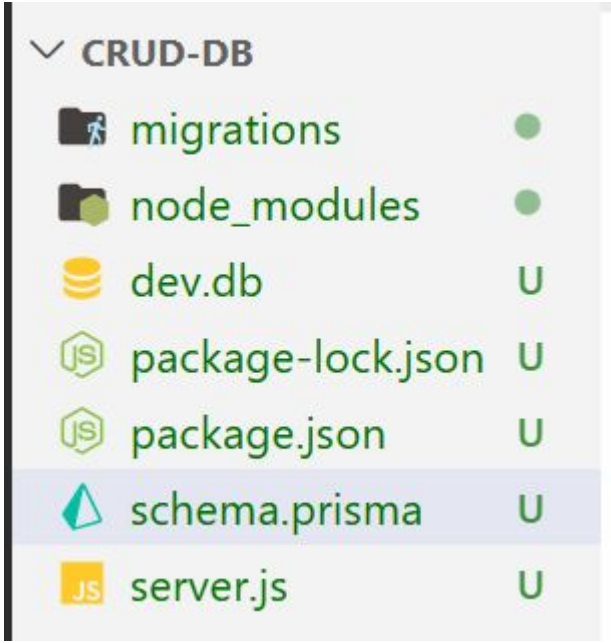
Não precisa decorar

- Documentação oficial
- Bookmarks
- Praticar principais métodos (CRUD)

Prisma ORM

Estrutura básica

schema.prisma



schema.prisma

```
// This is your Prisma schema file,
// learn more about it in the docs: https://pris.ly/d/prisma-schema

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "sqlite"
  url      = "file:./dev.db"
}

model User {
  id    Int     @id @default(autoincrement())
  nome  String
  email String  @unique

  @@map("usuarios")
}
```

Conhecendo o Prisma (documentação)

Prisma Documentation

Working with Prisma gives you a best-in-class TypeScript ORM, a declarative database migration system, and a database with everything you need to get started.

Try out what Prisma has to offer with one command:

```
$ npx prisma init --db
```

- Type-safe (gera um *client* específico, do tipo do schema)
- queries otimizadas
- schema declarativo
- Boa experiência de desenvolvimento

Sessões importantes

1 - Getting started: tutorial passo a passo

2 - Schema reference: referência da sintaxe

3 - Client API: Como usar o Prisma Client

4 - Migrate: Sistema de migrações

5 - Examples: Projetos prontos para estudar

Schema.prisma

- Arquivo principal do Prisma
- Define:
  - tabelas
  - relacionamentos
  - índices
  - configurações do banco de dados
  - configurações do client

schema.prisma

schema.prisma

```
// This is your Prisma schema file,
// learn more about it in the docs: https://pris.ly/d/prisma-schema

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "sqlite"
  url      = "file:./dev.db"
}

model User {
  id    Int     @id @default(autoincrement())
  nome  String
  email String  @unique

  @@map("usuarios")
}
```

Instalando o Prisma

```bash
# Prisma CLI (desenvolvimento)
npm install prisma --save-dev

# Prisma Client (produção)
npm install @prisma/client
```

Inicializando o Prisma no projeto

```bash
npx prisma init
```

Cria a pasta “prisma”

Cria o arquivo schema.prisma

Cria um arquivo .env (vamos falar sobre isso)

Migrations

O que são e como lidar com eles usando o Prisma

- Scripts de versionamento de banco de dados, que permitem:

Evoluir o schema (criar novos campos, novas tabelas, etc) de forma controlada

Sincronizar mudanças entre ambientes

Manter histórico de alterações

Reverter mudanças quando necessário

Como aplicar mudanças em tabelas

O prisma compara o schema.prisma com o estado do seu banco atual

Quando ele encontra algo que mudou, realiza o seguinte processo:

- Gera o SQL necessário para atualizar o seu banco
- Cria um arquivo de migration automaticamente
- Aplica as mudanças

Logo: para atualizar um banco, basta atualizar o arquivo *schema.prisma*

Executando Migrations

```
npx prisma migrate dev
```

ATENÇÃO

Prisma realiza o **UPGRADE** do banco de dados automaticamente, através dos migrations

Mas não possui **DOWNGRADE**, é preciso realizar essa ação manualmente, seja com um SQL “manual” ou algum outro procedimento

Vamos tentar na prática